In Javascript, you can create a variable like this:

```
let age = 25;
age = "Hello";
```

→ In JS allows you to change the values numbers to string.
→ But sometimes unexpected errors in applications

Typescript :-

Typescript avoid these kinds of mistakes by allowing us to specify the type of data a variable should contain.

```
let age : number = 25;
age = "Hello"; → This is error
```

Why :- Means because we declared age has a Number, In typescript doesn't allow to assign to a string.

2. Datatypes :-

There are three important types.

i. string :- A string Means text or characters.

code :-

```
let Name : string[ ] = "abc";
console.log(name);
```

o/p :- abc

let → create a variable
Name → variable Name
string → tells typescript it must be text.

ii. Number :-

Number type is a Numerical values

Ex :- Age, salary, Marks

code :-

```
let Age : Number = 25;
let Salary : Number = 10000;
let Marks : Number = 90;
console.log(Age);
console.log(Salary);
console.log(Marks);
```

iii) Boolean → True and false

code :-

```
let isstudent : boolean : "true";
let isloggedin : boolean = "false";
console.log(isstudent);
console.log(isloggedin);
```

Forex :- Engine a login application.

```
Let isLoggedIn : Boolon = "True";
if (isLoggedIn) {
  console.log("welcome");
} else {
  console.log("please login");
}
```

Array in Typescript :- is used to store Multiple values inside a single variable

Ex:- In Javascript

```
let students = ["a", b, c, d, e]
```

In Typescript :-

```
let students : string[] = ["a", b, c, d, e]
```

Numbers

```
let studentMarks : Number[] = [30, ...
```

Boolean

```
let Result : Boolean[] = [true, false, ...
```

4. Object in Typescript :-

object stores the Multiple related properties together

JS :-

```
let student = {
    name = "a";
    age = "2";
    isActive = 'true';
};
```

TS :-

```
let student : {
    name : "string";
    age : "number";
    isactive : boolean;
} = {
    name = "b";
    age : 2;
    isactive : true
};
```

5. Any :-

some time you don't know what type of value a variable will contain. Typescript provides any.

```
let value : any = 100;
value = "Hello";
value = true;
```

* Interface :- Interface is a blueprint of an object

Ex:- create a student registration application.

```
let student : {
    name : "string";
    age : Number;
    email : "string";
    phone ? : Number;
} = {
    name = "abcd";
    age = "22";
let student1 : student = {
    name = "abcd";
    age = 22;
    email = "abcd@gmail.com"
};
let student2 = {
    name = "ab"
    age = 40;
    email = "ab@gmail.com"
};
```

Interface

```
Interface student = {
```

```
    age : 30,
    email : "ab@gmail.com"
    Phone? : 12345 6789
};
console.log(student1);
console.log(student2);
```

* **Generics :-**

Create a functions that returns whatever value you give it.

```
function getnumber(value : number) : number {
    return value;
}
console.log(getnumber(100));
```

} same as string, boolean

function → Create a function
getvalue → function name
<T> → Generic type place holder
value : T → Input accepts type T
: T → Returns type T
~~arg~~ arg : T[] Accepts an array containing T values.
T | undefined → Return a T value or undefined

First Generic function :-

```
function getvalue<T>(value : T) : T {
    return value;
```

Generic function with arrays :-

```
let Numbers = [10, 20, 30, 40, 50];
```

You want to create a function that returns the first element of an array

```
function getfirst<T>(arg : T[]) : T | undefined {
    return arg[0];
}
console.log(getfirst([10, 20, 30, 40]));
```

} Same as string, boolean

o/p :- 10

Multiple Generics (T and U)

Imagine you want a function that accepts:
* Student name → string
* Student age → Number

```
function DisplayData<T, U>(name : T, age : U) : void {
    console.log(name);
    console.log(age);
```

Generics with interface :-

```
const NumberBox: Box<number> = {
  value: 100
};
```

} same as string, boolean.

Create a two interfaces :-

```
Interface User {
  id: number;
  name: string;
}
Interface Product {
  id: Number;
  name: string;
  price: Number;
}
```

Create a generic API response interface :-

```
const userResponse: ApiResponse<User> = {
  success: true,
  data: {
    id: 1,
    name: "abc"
  }
  Message: "user fetched"
};
const productResponse: ApiResponse<Product> = {
  success: true,
  data: {
    id: 2,
    name: "cd",
    price: 200
  }
  Message: "product fetched"
};
```

Enums :-

A collection of named predefined values.

enum → keyword used to create an enum

OrderStatus → enum name

Pending, shipped ..... enum members

How to create an enum

```
enum OrderStatus {
  Pending,
  Shipped,
  Processing,
  Delivered,
  Cancelled
}
let status : OrderStatus = OrderStatus.Pending;
console.log(status);
```

It can take the values.

0
1
2
3
4
5

O/P :-

0 → Means pending.

Enum :

```
// 1. enum Role

enum Role {
    Admin = "ADMIN",
    Agent = "AGENT",
    customer = "CUSTOMER",
}

// 2. Ticket status enum

enum TicketStatus {
    Open = "OPEN",
    InProgress = "IN PROGRESS",
    Resolved = "RESOLVED",
    closed = "CLOSED"
}

// 3. Ticket priority enum

enum TicketPriority {
    low = "LOW",
    Medium = "MEDIUM",
    High = "HIGH",
    Critical = "CRITICAL"
}

// 4. user Interface

Interface User {
    id = number;
    name : string;
    role : Role;
}

// 5. Ticket Interface

Interface Ticket {
    id : number;
    title : string;
    description : string;
    status : TicketStatus
    priority : TicketPriority
    assigned TO : user;
}

// 6. Create a user

const agent : User = {
    role : Role.Agent
};

// 7. Create a ticket

const ticket: Ticket = {
    id: 101,
    title : "payment failed",
    discription : "Unable to complete
                   online payment
    Status : TicketStatus.Open
    Priority : TicketPriority.Critical
    assigned TO : agent
};

console.log(ticket);
```

Utility

types:- 1. Omit, pick and partial

1. Omit → Remove selected properties from a type
2. Pick → selected only the properties we need.
3. Partial → Makes all properties optinal.